

A Plan for C++23 Ranges

Document #: P2214R0
Date: 2020-10-14
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Conor Hoekstra
<conorhoekstra@gmail.com>
Tim Song
<t.canens.cpp@gmail.com>

Contents

[1 Introduction](#)

[2 View adjuncts](#)

[3 Views](#)

[3.1 Full view or not full view](#)

[3.2 The zip family](#)

[3.2.1 A tuple that is writable](#)

[3.2.2 A tuple that is readable](#)

[3.2.3 zip and zip_transform's value_type](#)

[3.2.4 enumerate's first range](#)

[3.2.5 Does adjacent mean 2 or n?](#)

[3.3 The group_by family](#)

[3.4 Monadic binds](#)

[3.4.1 flat_map](#)

[3.4.2 transform_maybe](#)

[3.5 The sliding family](#)

[3.6 The take/drop family](#)

[3.7 Generative factories](#)

[3.8 Other view adapters](#)

[3.9 Derivatives of transform](#)

[3.9.1 views::const](#)

[4 Algorithms](#)

[4.1 Algorithms that Output a Value \(Catamorphisms\)](#)

[4.1.1 std::accumulate → ranges::fold](#)

[4.1.2 ranges::reduce](#)

[4.1.3 ranges::transform_reduce and ranges::inner_product](#)

[4.2 Algorithms that Output a Range \(Anamorphisms\)](#)

[4.2.1 std::adjacent_difference → ranges::adjacent_transform](#)

[4.2.2 std::partial_sum → ranges::partial_fold and std::{in,ex}clusive_scan](#)

[4.2.3 transform_{in,ex}clusive_scan](#)

[4.3 Parallel Algorithms](#)

[5 Actions](#)

[6 Plan Summary](#)

[6.1 Tier 1](#)

[6.2 Tier 2](#)

§ 1 Introduction

When Ranges was merged into C++20 [P0896R4], it was knowingly incomplete. While it was based on the implementation experience in range-v3 [range-v3], only a small part of that library was adopted into C++20. The Ranges proposal was big enough already, a lot of the pieces were separable and so could be included later.

But now that the core of Ranges has been included, later has come and we have to figure out what to do for C++23. This is a particularly trying period in the committee's history with the global pandemic and lack of face-to-face meetings. But we do already have a plan for C++23 [P0592R4] which laid out the following priorities:

The priority order of handling material is thus:

1. Material that is mentioned in this plan.
2. Bug fixes, performance improvements, integration fixes for/between existing features, and issue processing.
3. Material that is not mentioned in this plan.

and

Where are nex-gen Ranges in this plan?

We could certainly entertain more Actions and Views in the realm of Ranges. Whether such material appears for standardization is a bit unknown at this point.

We believe that adding more functionality to Ranges is important, even if it would technically be a 3rd priority item (unless you consider the spirit of the 2nd priority to include integration with itself).

But there is a *lot* of outstanding functionality that could be added. And while we think all of it should eventually be added (having more algorithms and more views is always a good thing), we realize that this is may be too much even in the C++23 time frame. Rather than having one-off papers that propose one bit of functionality (as in [P1255R6], [P1894R0], or [P2164R1]), we think it's important to take a big picture view of what is out there and triage the various parts into three separate buckets:

1. Functionality that is really important for C++23. That which is frequently used and broadly applicable, and thus whose absence is frequently complained about.
2. Functionality that would be nice to have for C++23. Not as frequently needed as the first bucket, but still clearly want it in the standard - but more tolerable if this slips. Ideally C++23 anyway, but simply less critical.
3. Functionality that is less important or needs more design work.

This paper provides our opinion for how to categorize Ranges functionality into those three buckets. We go through, in turn: [views-adjacent functionality](#), [views](#), [algorithms](#), and [actions](#) (which do not exist in C++20).

§ 2 View adjuncts

C++20 Ranges, and the range-v3 that birthed it, isn't just a collection of loosely related views and algorithms. There's some important other functionality there.

One critical piece of missing functionality is `ranges::to` [P1206R1]. It's not a view, but it is often used as the terminal component of a view pipeline to create a new trailing range - to finally collect the results of the computation being constructed. This is a top priority and is sorely missing.

Another important piece of functionality is simply the ability to print views. In range-v3, views were printable, which made it easy to debug programs or to provide meaningful output. For instance, the following program using range-v3 happily compiles:

```
#include <range/v3/view/iota.hpp>
#include <range/v3/view/transform.hpp>
#include <range/v3/view/filter.hpp>
#include <iostream>

int main() {
    namespace rv = ranges::views;
    auto r = rv::iota(0, 20)
        | rv::filter([](int i) { return i % 2 == 0; })
        | rv::transform([](int i){ return i * i; });
    std::cout << r;
}
```

and prints `[0, 4, 16, 36, 64, 100, 144, 196, 256, 324]`.

Similarly, `fmtlib` supports printing ranges with `fmt::join`, which is slightly more tedious but is at least still a single line:

```
#include <range/v3/view/iota.hpp>
#include <range/v3/view/transform.hpp>
#include <range/v3/view/filter.hpp>
#include <fmt/format.h>

int main() {
    namespace rv = ranges::views;
    auto r = rv::iota(0, 20)
        | rv::filter([](int i) { return i % 2 == 0; })
        | rv::transform([](int i){ return i * i; });
    fmt::print("{} ", fmt::join(r, " "));
}
```

But neither the ability to stream views directly nor `fmt::join` are in C++20, so there is no direct way to print a range at all.

We think it's important that C++23 provides the ability to `format all the views`. Since these are all standard library types, it is difficult for the user to be able to actually do this themselves and it's frustrating to even have to.

§ 3 Views

C++20 included a bunch of views, but range-v3 has a whole lot more. Views, much like algorithms, are the kind of thing where it's just generally good to have more of them. The C++20 standard library has over 100 algorithms, but only 17 range adapters and view factories (excluding `all` and `ref`). We want more.

We'll start this section by enumerating all the adapters in range-v3 (and a few that aren't), noting their current status, and ranking them according to our view of their priority for C++23, before describing how we came up with such a ranking.

View	Current Status	Proposed Priority
<code>addressof</code>	range-v3	Not proposed

View	Current Status	Proposed Priority
adjacent	(not in range-v3)	Tier 1
adjacent_transform	(not in range-v3)	Tier 1
adjacent_filter	range-v3	Tier 3
adjacent_remove_if	range-v3	Tier 3
all	C++20	C++20
any_view<T>	range-v3	Not proposed
c_str	range-v3	Tier 3
cache1	range-v3	Tier 1, largely for flat_map. Possibly renamed as cache_last or cache_latest
cartesian_product	range-v3	Tier 1
chunk	range-v3	Tier 1
common	C++20	C++20
concat	range-v3	Tier 2
const_	range-v3	Tier 3
counted	C++20	C++20
cycle	range-v3	Tier 2
delimit	range-v3	Tier 2
drop	C++20	C++20
drop_last	range-v3	Tier 2
drop_last_while	(not in range-v3)	Tier 2
drop_exactly	range-v3	Tier 3
drop_while	C++20	C++20
empty	C++20	C++20
enumerate	range-v3	Tier 1
filter	C++20	C++20
for_each	range-v3	Tier 1. Most languages call this flat_map, but we probably need to call it transform_join. Also allow for non-views
generate	range-v3	Tier 2
generate_n	range-v3	Tier 2
group_by	range-v3	Tier 1 (but not how range-v3 does it). Possibly renamed to chunk_by.. Also consider a variant chunk_on as Tier 2
head	(not in range-v3)	Tier 3
indirect	range-v3	Not proposed
intersperse	range-v3	Tier 2

View	Current Status	Proposed Priority
ints	range-v3	Unnecessary unless people really hate iota.
iota	C++20	C++20
join	partially C++20, lacks delimiter ability	Tier 1 (adding delimiter ability via <code>join_with</code>)
keys	C++20	C++20
linear_distribute	range-v3	Tier 3
maybe	proposed in [P1255R6]	???
move	range-v3	Not proposed
partial_sum	range-v3	Tier 2, but not taking a callable (solely as a specialized form of <code>scan</code>)
remove	range-v3	Tier 2
remove_if	range-v3	Tier 2
repeat	range-v3	Tier 2
repeat_n	range-v3	Tier 2
replace	range-v3	Tier 2
replace_if	range-v3	Tier 2
reverse	C++20	C++20
sample	range-v3	Tier 3
scan	(not in range-v3)	Tier 2, as a rename of what is <code>partial_sum</code> in range-v3
single	C++20	C++20
slice	range-v3	Tier 3
sliding	range-v3	Tier 1, renamed to <code>slide</code>
split	C++20, but unergonomic	See [P2210R0] .
split_when	range-v3	Tier 2
stride	range-v3	Tier 1
tail	range-v3	Tier 3
take	C++20	C++20
take_exactly	range-v3	Tier 3
take_last	range-v3	Tier 2
take_last_while	(not in range-v3)	Tier 2
take_while	C++20	C++20
tokenize	range-v3	Not proposed
transform_maybe	(not in range-v3)	Tier 1, as a more ergonomic maybe (Haskell calls this <code>mapMaybe</code> , Rust calls this <code>filter_map</code>)
trim	range-v3	Tier 2
unbounded	range-v3	Not proposed

View	Current Status	Proposed Priority
unique	range-v3	Tier 2
values	C++20	C++20
zip	range-v3	Tier 1
zip_with	range-v3	Tier 1, renamed to zip_transform

§ 3.1 Full view or not full view

One question we have to ask is which of these views need to be full-blown view types and which do not. This is an important question to deal with in light of the very real limitation that is LWG’s bandwidth. Views that need to be full types necessitate more longer and more complex specification which require more LWG time. Views that need *not* be full types could save some time.

Consider `views::tail`. This is a very simple range adapter that simply drops its first element. As such, it could be specified entirely as:

```
namespace std::ranges::views {
    inline constexpr auto tail = drop(1);
}
```

The above is a semantically correct implementation. However, is it the implementation we would actually want for `views::tail`? There are a few extra things `views::drop` has to do. `views::drop` must store a count and a cache for the begin iterator for forward ranges (to satisfy `0(1)`), but `views::tail` has to do none of these things since 1 is a constant and invoking `next()` one time still satisfies `0(1)`.

This puts us in an interesting position: we either adopt a known suboptimal implementation of `tail` with minimal LWG cost (such that we could likely adopt this for C++23) or we could hold off for the optimal implementation (in which case we could not in good conscience put `tail` as a Tier 1 view, as it is certainly not that important). As such, we have tentatively marked it as Tier 3.

Take a different view, unbounded. A perfectly valid implementation of it, in its entirety is:

```
namespace std::ranges::views {
    inline constexpr auto unbounded = [](input_iterator auto it){
        return subrange(std::move(it), unreachable_sentinel);
    };
}
```

Unlike `tail`, there isn’t really some other, better implementation here. Maybe we’d prefer to give `unbounded_view<I>` its own type rather than piggy-backing off of `subrange`, but it wouldn’t really affect the choice of functionality overall. Here the question is less, do we do it fast for C++23 or do it right for C++26 or C++20. Rather, the question is more: do we even need this at all? We think we don’t.

Consider two different, closely related views: `views::zip` and `views::zip_transform` (which range-v3 calls `zip_with`). Each could potentially be specified in terms of the other:

- `views::zip(Es...)` could be `views::zip_transform(forward-as-tuple, Es...)`
- `views::zip_transform(F, Es...)` could be `views::zip(Es...) | views::transform(applied(F))`

where `forward-as-tuple` is a function object version of `std::forward_as_tuple` and `applied(F)(x)` is equivalent to `std::apply(F, x)`.

But they're not *quite* equivalent — they differ in their handling of ranges that produce prvalues. `zip_transform` can't differentiate between prvalues and xvalues, while `zip` would need to, so we lose that distinction by implementing `zip` in terms of `zip_transform`.

Likewise, a first-class `zip_transform` would construct those prvalues directly into the parameters of the function that gets passed in; whereas if we implemented `zip_transform` in terms of `zip` and `transform` those prvalues would have to first be materialized into a tuple and then moved into the function.

Even though these two views are very, very similar to each other, we still don't want to specify one in terms of the other even if it means more specification effort. And we still propose that both be Tier 1 views due to their overall importance. It would be better to adopt `zip` on its own and provide a new function adapter *applied*, leaving the door open to a better `zip_transform` in the future, than specify `zip_transform` in terms of `zip` and `transform`.

Another interesting example is `transform_maybe`. As we'll see later, `transform_maybe(E, F)` can easily be specified in terms of three adapters today (a `transform` followed by a `filter` followed by a `transform`). But while `tail` can be made a little bit better as a standalone view, `transform_maybe` can be *substantially* improved.

Ultimately, this question of full view or not full view is one that should guide review of this paper.

§ 3.2 The `zip` family

The `zip` family of range adapters is an extremely useful set of adapters with broad applicability. It consists of five user-facing range adapters:

1. `zip`
2. `zip_transform` — `zip_with` in range-v3
3. `enumerate` — `enumerate(E)` is [roughly equivalent](#) to `zip(iota(0), E)`
4. `adjacent` — `adjacent(E)` is equivalent to `zip(E, E | drop(1))`
5. `adjacent_transform` — is to `adjacent` what `zip_transform` is to `zip`

We hope to be able to specify these five in terms of two, exposition-only range adapters: *iter-zip-transform*<V> and *iter-adjacent-transform*<V>.

Indeed, we have many algorithms that exist largely because we don't have `zip` (and prior to Ranges, didn't have the ability to compose algorithms). We have several algorithms that have single-range-unary-function and a two-range-binary-function flavors. What if you wanted to `ranges::transform` 3 ranges? Out of luck. But in all of these cases, the two-range-binary-function flavor could be written as a single-range that is a `zip` of the two ranges, `adjacent_difference` is `adjacent_transform`, `inner_product` is `zip_transform` followed by `accumulate`, and so on and so on. This is why we think `zip` is the top priority view.

range-v3 uses the name `zip_with`, but here we are instead proposing `zip_transform`. The reason is that what we're doing is, quite simply, a `transform` and in C++, we don't really have a strong association of that with the `_with` suffix. Instead, we have algorithms like `begins_with` and concepts like `swappable_with` and `totally_ordered_with`, which have nothing to do with this sort of thing.

The most generic of this group is *iter-zip-transform*<V>. It is core functionality for the family and aids significantly in the specification effort where LWG bandwidth is concerned. *iter-zip-transform*(f, rs...) takes an n-ary invocable and n ranges and combines those into a single range whose values are the result of `f(is...)`, where the `is` are the iterators into those ranges (note: iterators, not what they refer to). The size of an *iter-zip-transform* is the minimum size of its ranges. The reference type is the type of `f(*is...)` while the `value_type` is V (described in more detail [later](#) along with the specific choices for `Vzip_transform` and `Vzip` used below).

From *iter-zip-transform*, we get:

- `zip_transform(F, Es...)` is expression-equivalent to `iter-zip-transform<Vzip_transform>(indirected(F), Es...)`, where *indirected* is a specification-only function object that dereferences all of its arguments before invoking *F* (e.g. *indirected(F)(x, y, z)* is equivalent to *F(*x, *y, *z)*)
- `zip(Es...)` is expression-equivalent to `views::iter-zip-transform<Vzip>(make-zip-tuple, Es...)`, where *make-zip-tuple* is basically `[]<input_iterator... Is>(Is... is) { return tuple<iter_reference_t<Is>...>(*is...); }`.
- `enumerate(E)` is expression-equivalent to `zip(index-view(E), E)`. We will discuss why *index-view(E)* instead of *iota(range_size_t<decltype(E)>(0) later*.

But we do not want to define `adjacent(E)` as `zip(E, E | drop(1))`. The primary reason to avoid doing this, and to make `adjacent` (or rather, *iter-adjacent-transform<V>*) a first-class view type despite the added specification effort, is that it allows supporting move-only views. On top of that, it has the benefit of not having to store the view twice or the other cruft that `drop` has to store (see also the `drop/tail` discussion earlier).

Once we have *iter-adjacent-transform<V>*, `adjacent` and `adjacent_transform` follow in the same way that we defined `zip` and `zip_transform`.

In short, we get five extremely useful ranges for the price of three specification-only ones. Which is why we think they should be considered and adopted as a group.

But in order to actually adopt `zip` and friends into C++23, we need to resolve several problems.

§ 3.2.1 A tuple that is writable

Consider the following:

```
std::vector<int> vi = /* ... */;
std::vector<std::string> vs = /* ... */;
ranges::sort(views::zip(vi, vs));
```

Does this operation make sense? Definitely! It would be very useful if this compiled. Unfortunately, as-is, it will not. We need to go back and understand *why* this won't compile today and what we need to do to fix it.

In [\[range-v3.573\]](#), user Jarod42 pointed out that the following program used to compile, yet provide a non-sensical answer:

```
struct C
{
    explicit C(std::string a) : bar(a) {}

    std::string bar;
};

int main()
{
    std::vector<C> cs = { C("z"), C("d"), C("b"), C("c") };

    ranges::sort(cs | ranges::view::transform([](const C& x) {return x.bar;}));

    for (const auto& c : cs) {
        std::cout << c.bar << std::endl;
    }
}
```


The range was *not* sorted, the emitted output was still in the same order as the input... because we're sorting a range of `prvalue std::strings`, and trying to swap `prvalue std::strings` makes little sense because it doesn't do anything.

But the reason it compiled was that the constraints checked if the iterators were assignable-through, which in this case was equivalent to checking if `std::string() = std::string()` is a valid expression... and it, unfortunately, is. Assignment operators simply are not reference-qualified - they could not have been for a long time, and they are not now. The question became then - how can the library ensure that the above nonsense *does not compile*?

As discussed in [\[std2.381\]](#):

One fix would be to require that `*o` return a true reference, but that breaks when `*o` returns a proxy reference. The trick is in distinguishing between a `prvalue` that is a proxy from a `prvalue` that is just a value. The trick lies in recognizing that a proxy always represents a (logical, if not physical) indirection. As such, adding a `const` to the proxy should not effect the mutability of the thing being proxied. Further, if `decltype(*o)` is a true reference, then adding `const` to it has no effect, which also does not effect the mutability. So the fix is to add `const` to `decltype(*o)`, `const_cast *o` to that, and then test for writability.

Which is how we ended up with the `indirectly_writable` (the concept formerly known as `writable`) requiring `const`-assignability.

Hence, in order to make `ranges::sort(zip(vi, vs))` compile, we need to make `zip_view<R...>::iterator` model `indirectly_writable`, which means we need to make `std::tuple` `const`-assignable. That is, adding the following assignment operators, appropriately constrained:

```
// [tuple.assign], tuple assignment
constexpr tuple& operator=(const tuple&);
+ constexpr tuple& operator=(const tuple&) const;
constexpr tuple& operator=(tuple&&) noexcept(see below);
+ constexpr tuple& operator=(tuple&&) const noexcept(see below);

template<class... UTypes>
constexpr tuple& operator=(const tuple<UTypes...>&);
+ template<class... UTypes>
+ constexpr tuple& operator=(const tuple<UTypes...>&) const;
template<class... UTypes>
constexpr tuple& operator=(tuple<UTypes...>&&);
+ template<class... UTypes>
+ constexpr tuple& operator=(tuple<UTypes...>&&) const;
```

Or, rather than change `std::tuple`, we can have `zip_view<R...>::value_type` and `zip_view<R...>::reference` be an entirely different tuple type than `std::tuple`, introducing a second tuple type into the standard library: `std2::tuple2`. This just seems like a facially terrible idea, we already have one tuple type, we should just have it solve this problem for is.

We therefore propose that `std::tuple<T...>` be `const`-assignable whenever all of `T...` are `const`-assignable. And likewise for `std::pair<T, U>`, for consistency, as well as the other proxy reference types in the standard library: `std::vector<bool>::reference` and `std::bitset<N>::reference`.

§ 3.2.2 A tuple that is readable

We encourage everyone to review Eric Niebler's four-part series on iterators [\[niebler.iter.0\]](#) [\[niebler.iter.1\]](#) [\[niebler.iter.2\]](#) [\[niebler.iter.3\]](#), as this problem is covered in there in some depth.

There is a fundamental problem with standard library algorithms and how they interact with proxy iterators. One example used in the series is `unique_copy`:

```

1 // Copyright (c) 1994
2 // Hewlett-Packard Company
3 // Copyright (c) 1996
4 // Silicon Graphics Computer Systems, Inc.
5 template <class InIter, class OutIter, class Fn,
6          class _Tp>
7 OutIter
8 __unique_copy(InIter first, InIter last,
9              OutIter result,
10              Fn binary_pred, _Tp*) {
11     _Tp value = *first;
12     *result = value;
13     while (++first != last)
14         if (!binary_pred(value, *first)) {
15             value = *first;
16             *++result = value;
17         }
18     return ++result;
19 }

```

[...] Note the value local variable on line 11, and especially note line 14, where it passes a value and a reference to `binary_pred`. Keep that in mind because it's important!

[...] Why do I bring it up? Because it's *super problematic* when used with proxy iterators. Think about what happens when you try to pass `vector<bool>::iterator` to the above `__unique_copy` function:

```

std::vector<bool> vb{true, true, false, false};
using R = std::vector<bool>::reference;
__unique_copy(
    vb.begin(), vb.end(),
    std::ostream_iterator<bool>{std::cout, " "},
    [](R b1, R b2) { return b1 == b2; }, (bool*)0 );

```

This *should* write a “true” and a “false” to `cout`, but it doesn't compile. Why? The lambda is expecting to be passed two objects of `vector<bool>`'s proxy reference type, but remember how `__unique_copy` calls the predicate:

```
if (!binary_pred(value, *first)) { /*...*/
```

That's a `bool&` and a `vector<bool>::reference`. Ouch!

The blog goes on to point out that one way to resolve this is by having the lambda take both parameters as `auto&&`. But having to *require* a generic lambda is a bit... much. This was the reason we have the idea of a `common_reference`. Rewriting the above to be:

```

using R = std::iter_common_reference_t<std::vector<bool>::reference>;
__unique_copy(
    vb.begin(), vb.end(),
    std::ostream_iterator<bool>{std::cout, " "},
    [](R b1, R b2) { return b1 == b2; }, (bool*)0 );

```

This now works. And that is now the requirement that we have for iterators, that they be `indirectly_readable` - which, among other things, requires that there be a `common_reference` between the iterator's reference type and the iterator's `value_type`:

```

template<class In>
concept indirectly_readable_impl =
    requires(const In in) {
        typename iter_value_t<In>;
        typename iter_reference_t<In>;
    };

```

```

    typename iter_rvalue_reference_t<In>;
    { *in } -> same_as<iter_reference_t<In>>;
    { ranges::iter_move(in) } -> same_as<iter_rvalue_reference_t<In>>;
} &&
common_reference_with<iter_reference_t<In>&&, iter_value_t<In>&> &&
common_reference_with<iter_reference_t<In>&&, iter_rvalue_reference_t<In>&&> &&
common_reference_with<iter_rvalue_reference_t<In>&&, const iter_value_t<In>&>;

template<class In>
concept indirectly_readable =
    indirectly_readable_impl<remove_cvref_t<In>>;

template<class I>
concept input_iterator =
    input_or_output_iterator<I> &&
    indirectly_readable<I> &&
    requires { typename ITER_CONCEPT(I); } &&
    derived_from<ITER_CONCEPT(I), input_iterator_tag>;

```

How does this relate to zip?

Letting `I` be `zip_view<R...>::iterator` for some set of ranges `R...`, it just follows from first principles that `iter_reference_t<I>` should be `std::tuple<range_reference_t<R>...>` — dereferencing a zip iterator should give you a tuple of dereferencing all the underlying ranges' iterators. And then it sort of follows from symmetry that `iter_value_t<I>` should be `std::tuple<range_value_t<R>...>`. We'll discuss this in more depth [in a later section](#).

But then what does `iter_common_reference_t<I>` end up being?

Let's pick some concrete types. Taking our earlier example of:

```

std::vector<int> vi = /* ... */;
std::vector<std::string> vs = /* ... */;
ranges::sort(views::zip(vi, vs));

```

We have a value type of `std::tuple<int, std::string>` and a reference type of `std::tuple<int&, std::string&>`. The common reference of those exists: the reference type is convertible to the value type but not in the other direction, so the common reference is just the value type of `std::tuple<int, std::string>`. This might be odd, having a common *reference* type that has no reference semantics, but it does work. And in some cases you can't really do better (as in the `vector<bool>` example from the blog series, the common reference is `bool`).

But where we really run into problems is with non-copyable types. If instead of zipping a `std::vector<int>` and a `std::vector<std::string>`, we changed the first range to be a `std::vector<std::unique_ptr<int>>`, what happens? We have a value type of `std::tuple<std::unique_ptr<int>, std::string>` and a reference type of `std::tuple<std::unique_ptr<int>&, std::string&>`, similar to before. But now, neither is convertible to the other.

The constructor overload set of `std::tuple` is... intense to say the least. But the relevant ones are:

```

1 | constexpr explicit(see below) tuple();
2 | constexpr explicit(see below) tuple(const Types&...);           // only if sizeof...
   |   (Types) >= 1
3 | template<class... UTypes>
4 |   constexpr explicit(see below) tuple(UTypes&&...);           // only if sizeof...
   |   (Types) >= 1
5 |
6 | tuple(const tuple&) = default;
7 | tuple(tuple&&) = default;
8 |
9 | template<class... UTypes>
10 |   constexpr explicit(see below) tuple(const tuple<UTypes...>&);
11 | template<class... UTypes>

```

```

12     constexpr explicit(see below) tuple(tuple<UTypes...>&&);
13
14 template<class U1, class U2>
15     constexpr explicit(see below) tuple(const pair<U1, U2>&);    // only if sizeof...
    (Types) == 2
16 template<class U1, class U2>
17     constexpr explicit(see below) tuple(pair<U1, U2>&&);        // only if sizeof...
    (Types) == 2

```

We have a converting constructor from `std::tuple<U...> const&`, viable if `(constructible_from<T, U const&> && ...)`, and a converting constructor `std::tuple<U...>&&`, viable if `(constructible_from<T, U&&> && ...)`. In both cases, we're just distributing the qualifiers.

When trying to construct a `std::tuple<std::unique_ptr<int>, std::string>` from a `std::tuple<std::unique_ptr<int>&, std::string&>`, we reject the converting constructor of lines 9-10 because we can't construct a `std::unique_ptr<int>` from a `std::unique_ptr<int> const&`. `std::unique_ptr` isn't copyable, and that's not going to change.

When trying to construct a `std::tuple<std::unique_ptr<int>&, std::string&>` from an *lvalue* `std::tuple<std::unique_ptr<int>, std::string>` (the fact that it's an lvalue is important), we reject that very same converting constructor because we can't construct a `std::unique_ptr<int>&` from a `std::unique_ptr<int> const&`. Can't bind a non-const lvalue reference to a const lvalue. But we can of course bind a non-const lvalue reference to a non-const lvalue.

And indeed that's how close this is to working. All we need is one more constructor (although we added two here for completeness):

```

1  constexpr explicit(see below) tuple();
2  constexpr explicit(see below) tuple(const Types&...);          // only if sizeof...
    (Types) >= 1
3  template<class... UTypes>
4      constexpr explicit(see below) tuple(UTypes&&...);          // only if sizeof...
    (Types) >= 1
5
6  tuple(const tuple&) = default;
7  tuple(tuple&&) = default;
8
9  + template<class... UTypes>
10 +     constexpr explicit(see below) tuple(tuple<UTypes...>&);
11     template<class... UTypes>
12         constexpr explicit(see below) tuple(const tuple<UTypes...>&);
13     template<class... UTypes>
14         constexpr explicit(see below) tuple(tuple<UTypes...>&&);
15 + template<class... UTypes>
16 +     constexpr explicit(see below) tuple(const tuple<UTypes...>&&);
17
18     template<class U1, class U2>
19         constexpr explicit(see below) tuple(const pair<U1, U2>&);    // only if sizeof...
    (Types) == 2
20     template<class U1, class U2>
21         constexpr explicit(see below) tuple(pair<U1, U2>&&);        // only if sizeof...
    (Types) == 2

```

If only we had a way to express “a forwarding reference to `tuple<UTypes...>`” in the language. But if we add these constructors, then suddenly we *can* construct a `std::tuple<std::unique_ptr<int>&, std::string&>` from an lvalue `std::tuple<std::unique_ptr<int>, std::string>`. And that would just end up binding the references as you would expect.

Such a change to the constructor set of `std::tuple` means that all of our `zip_view` iterators can actually be `indirectly_readable`, which means they can actually count as being iterators. In all cases then, the common reference type of zip iterators would become the reference type. Indeed, this even fixes the issue we mentioned earlier - where even when our underlying types were copyable, we originally ended up with a common reference type of `std::tuple<int, std::string>`, a type that does not have reference semantics. But now it would have a common reference type of `std::tuple<int&, std::string&>`, which certainly has reference semantics.

We therefore propose that to extend the constructor overload set of `std::tuple<T...>` to add converting constructors from `std::tuple<U...&>` and `std::tuple<U...> const&&`. And likewise for `std::pair<T, U>`, for consistency.

§ 3.2.3 zip and zip_transform's value_type

Returning to our favorite example again:

```
std::vector<int> vi = /* ... */;
std::vector<std::string> vs = /* ... */;

auto a = views::zip(vi, vs);
```

Let's talk about `a`. The reference type of `a` is `std::tuple<int&, std::string&>`. That's really the only option, it's the whole point of `zip::range_reference_t<zip_view<R...>>` needs to be `std::tuple<range_reference_t<R>...>`.

But what's the `value_type` of `a`? We ideally want the `value_type` to be something without reference semantics, something that we can properly hold onto and have a real copy. For a lot of ranges, the reference type is some `T&` or `T const&` and so the `value_type` is just `T`. But here, our reference is a `prvalue`. So we have a choice.

We could do `std::remove_cvref_t` on the reference as usual, that would give us just reference back, so the `value_type` of `a` would be `std::tuple<int&, std::string&>`. Not really an independent, value type.

Or, since we know what `zip` is, we don't have to guess what a good `value_type` might be. We could use the ranges themselves to tell us what that is. Each constituent `R` already provides a `range_value_t<R>`, and if those choices are good enough for the `Rs`, they should be good enough for `zip`. That is, `std::tuple<range_value_t<R>...>`. Which, for `a`, would be `std::tuple<int, std::string>`.

But there's a wrinkle here, consider:

```
auto b = views::zip_transform([](auto&... r){
    return std::tie(r...);
}, vi, vs);
```

The reference type of `b` is necessarily determined by the callable here: `invoke_result_t<F&, range_reference_t<R>...>`. In this case, this gives us `std::tuple<int&, std::string&>`. Notably, this is exactly the same reference type as with `saw` with `a`.

But what's the `value_type` of `b`? Unlike in the case of `zip`, with `zip_transform` we really have no idea where the reference type came from. It need not be any kind of reference at all, it could be... `int`. For `zip_transform`, we really can't do much better than `remove_cvref_t<invoke_result_t<F&, range_reference_t<R>...>>` — which again gets us back to the same `std::tuple<int&, std::string&>`.

A hypothetical different direction would be to introduce a type trait that would inform range adapters how to turn a reference-semantic type to a value type. Something that would turn `std::vector<bool>::reference` into `bool`, or `std::tuple<T&...>` into `std::tuple<T...>`. Then other range adapters could have used it themselves.

But `views::transform` already exists, and is very similar in nature to `views::zip_transform`. Indeed, `views::zip_transform(F, E...)` is very nearly `views::zip(E...) | views::transform([=](auto&& tup){`

`return std::apply(F, std::forward<decltype(tup)>(tup)); }`), so any attempt to make `zip_transform`'s `value_type` line up with `zip`'s would mean changing the existing behavior of `views::transform`. That makes it unviable to pursue (assuming it were even a good idea to begin with).

The other way to get `zip_transform`'s `value_type` to be consistent with `zip`'s is to abandon the aforementioned symmetry and just use `remove_cvref_t` to determine `zip`'s `value_type` as well. But we find this questionable — we shouldn't sacrifice `zip` at the altar of consistency. It's really not that big of a deal, it's an inconsistency that only really surfaces when users would use `zip_transform` to do exactly what `zip` already does. But why would they do that, when they have `zip`?

In short, we propose that:

	reference	value_type
<code>zip_view<R...></code>	<code>std::tuple<range_reference_t<R>...></code>	<code>std::tuple<range_value_t<R>...></code>
<code>zip_transform_view<F, R...></code>	<code>invoke_result_t<F&, range_reference_t<R>...></code>	<code>remove_cvref_t<invoke_result_t<F&, range_reference_t<R>...>></code>

Or for this specific example:

	reference	value_type
<code>a — zip(vi, vs)</code>	<code>std::tuple<int&, std::string&></code>	<code>std::tuple<int, std::string></code>
<code>b — zip_transform(std::tie, vi, vs)</code>	<code>std::tuple<int&, std::string&></code>	<code>std::tuple<int&, std::string&></code>

We think these would be the most valuable choices.

As such, we can be more precise in our earlier formulation and say that:

- `views::zip_transform(F, E...)` is expression-equivalent to `views::iter_zip_transform<remove_cvref_t<invoke_result_t<decltype(F)&, range_reference_t<decltype(E)>...>>>(indirected(F), E...)`
- `views::zip(E...)` is expression-equivalent to `views::iter_zip_transform<std::tuple<range_value_t<decltype(E)>...>>(forward-as-tuple, E...)`

and similar for `adjacent_transform` and `adjacent`.

§ 3.2.4 enumerate's first range

We wrote earlier that `enumerate(r)` can be defined as `zip(iota(range_size_t<R>(0)), r)`, but it turns out that this isn't *quite* right, and we need to do something slightly different.

It turns out `iota` is a fairly complicated view due to wanting to avoid undefined behavior in its `difference_type` [P1522R1]. `iota_view` has to choose a type that is wide enough to properly compute differences. One of the important points in that paper is that integer-like types are:

Not required to be weaklyIncrementable: There is no requirement that a user-defined integer-like type specify what *its* difference type is. (If we were to require that, then by induction implementors would be on the hook to provide an infinite-precision integer-like type.) Therefore, `iota_view<iter_difference_t<I>>` for an arbitrary iterator `I` may in fact be ill-formed.

Similarly, `iota(range_size_t<R>(0))` need not be valid [range-v3.1141]. And, even if it were valid, it may be a wider integer type than would be strictly necessary. But `enumerate` is not as general-purpose as `iota`. From Eric Niebler:

However, the `enumerate` view is special. The `iota_view<diffmax_t>` that `view::enumerate` constructs will never produce a number less than zero or greater than the maximum `diffmax_t`. So `diffmax_t` itself has enough bits to be usable as the difference type of `iota_view<diffmax_t>::iterator`.

I can solve this problem with a custom `index_view` that doesn't try to promote the index type when computing the difference type.

And this is what `range-v3` does: `enumerate(r)` is defined as `zip(index-view<range_size_t<R>, range_difference_t<R>>(), r)`, where `index-view<S, D>` is basically a specialized version of `iota` such that `range_size_t<index-view<S, D>>` is `S` and `range_difference_t<index-view<S, D>>` is `D`. That is, rather than trying to compute a size and difference types to fit the range, we just use the provided range's size and difference types. If it's good enough for `R`, it's good enough for `enumerate_view<R>!`

This `index-view` can be exposition-only, it's effectively an implementation detail of `enumerate`.

§ 3.2.5 Does adjacent mean 2 or n?

In C++20, we have the range adapters `keys` and `values` which are actually specified as `elements<0>` and `elements<1>`, respectively, where `elements<N>` is a generic range adapter that picks the `N`th part from a range of tuples.

Now consider `adjacent`. This view could simply mean to take elements two-at-a-time, taking a range of `T` and producing a range of `(T, T)` (as a tuple). But there isn't anything particularly special about two, just that it'd probably be the most used value. There's not much additional specification effort to create a range adapter that takes elements `n`-at-a-time and produces a range of `n`-tuple. In that case, we could specify `adjacent` as `slide_as_tuple<2>` (this closely resembles the algorithm `slide` described [later](#), the difference being that here we need the value as a constant expression to produce a range of tuples, while there we produce a range of ranges). Alternatively, `adjacent` itself could be the generic algorithm and we could introduce a specialized name to mean `adjacent<2>`. For instance, Python calls this `pairwise` and Kotlin calls it `zipWithNext`.

§ 3.3 The group_by family

This family, as the name suggests, take a range of `T` and groups them based on some provided function. `group_by` is one of the most consistently named algorithms across all languages (modulo choice of spelling, as in `groupBy` or `group-by`) yet there are actually three-ish different approaches to what this algorithm actually means:

1. Take a binary predicate, `(T, T) -> bool`, and invoke this predicate on consecutive elements and start a new group when that predicate returns `false`. Using Haskell notation, we're taking a `[T]` (range of `T`) and producing a `[[T]]` (range of range of `T`).
2. Take a unary function, `T -> U` (such that `U` models `equality_comparable`), and group consecutive elements with the same `U`. Following the law of useful return, these algorithms don't just give you a `[[T]]` back, they rather give you a `[(U, [T])]` — the algorithm had to compute the `U`s for each element so they should give it to the user.
3. Take a unary function, `T -> U`, and return a dictionary that maps every `U` to a list of `T`s that mapped to it.

Haskell, Elixir, D (`chunkBy`), and `range-v3` (~ish) provide the first kind. Rust, Python, D (also `chunkBy` — it allows both uses), and F# provide the second. Clojure, Kotlin, and Scala provide the third.

Swift recently announced its own set of algorithms [swift.algorithms]. It provides the first kind as `chunked(by: f)` and the second kind as `chunked(on: f)` (Swift allows overloading based on named function arguments).

range-v3 is mostly in the first category, except its implementation choice is quite different from the other languages. It always compares the *first* element in each subrange with each subsequent one, while the others always compare *consecutive* elements. As such, they may yield very different results:

```
>>> groupBy (<=) [1,2,2,3,1,2,0,4,5,2]
[[1,2,2,3],[1,2],[0,4,5],[2]] -- Haskell's implementation
[[1,2,2,3,1,2],[0,4,5,2]]    -- range-v3's implementation
```

In Haskell, the second `1` starts a new group because we're comparing it with its previous element and `3 <= 1` is `false`. But in range-v3, the second `1` does not start a new group because we're comparing it with the first element of the group, and `1 <= 1` is `true`. We think the Haskell/Elixir/D choice is more familiar and more useful.

The question is *which* one of the three options would we want to pick for C++Next?

Well, when it comes to algorithms, the more the better. We think it's clear we wouldn't want to pick the 3rd option (we can easily produce a dictionary from the 2nd option), so the question is between the first two. While the binary version of `group_by` is more generic (since you approximate the unary version in terms of it, even it's a different shape) and thus more broadly applicable, the unary version also comes up frequently and as such we feel that we should provide both.

We could hypothetically follow the D model and provide both under the same name, selecting based on whether the provided callable is a unary invocable or a binary predicate — but we're not sure if that's the best idea.

Given the time and bandwidth pressure, we think the top priority is the binary `group_by` and the unary form, while still very useful, should be a Tier 2 addition. As far as naming goes, there two approaches we could take:

- `group_by` takes a binary predicate and either `group_on` or `group_by_key` takes a unary transform.
- `chunk_by` takes a binary predicate and `chunk_on` takes a unary transform.

The latter choice also lines up with a different algorithm that takes a range and yields a range of ranges: [chunk](#).

§ 3.4 Monadic binds

We added `views::transform` in C++20, but there are closely related views in that family, which several other languages also provide.

`views::transform` takes a range of `T` and a function `T -> U` and produces a range of `U`. But we can play around with the shape of the callable to produce two other extremely useful adapters:

- we can take a function `T -> RangeOf<U>` and produce a range of `U`. That is, take the resulting range and flatten it out into a single range.
- we can take a function `T -> optional<U>` and produce a range of `U` from those resulting optionals that are engaged.

The former is commonly known as `flat_map` (because it's a `map` followed by a `flatten`), although C++20's version of `flatten` is named `join` and C++20's version of `map` is named `transform`. So perhaps this adapter should be named `join_transform` or `transform_join`? Eughh?

The latter is called `filter_map` in Rust and `compactMap` in Swift. Neither strike us as great names either. A Haskell package calls this `mapMaybe` which informs our proposed name: `transform_maybe`.

There really aren't any particular thorny library issues to resolve here, simply a question of specification.

§ 3.4.1 flat_map

`flat_map(E, F)` is very nearly `E | transform(F) | join`. Very nearly, because that doesn't *quite* work. If the callable returns a prvalue range that is not a view (a seemingly specific constraint that is actually a very common use-case - consider a function returning a vector<int>), the above doesn't work. This specific case frequently comes up on StackOverflow asking for workarounds.

And there is one in range-v3, it's called `cache1` — though here we suggest renaming it as `cache_latest`. As the name suggests, it caches a single element at a time from its parent view - which allows the range to be joined. With the adoption of `cache_latest` (a view itself with other broad applicability), we could specify `views::flat_map(E, F)` as expression-equivalent to:

- `E | views::transform(F) | views::join` if that is a valid expression.
- Otherwise, `E | views::transform(F) | views::cache_latest | views::join`.

range-v3 has `cache1` yet only supports the first bullet, under the name `views::for_each`.

Despite being composed of other adapters that we already have, this is sufficiently complex to implement, sufficiently important, and requires a new adapter to boot, that it merits Tier 1 inclusion. Unlike other examples we've seen in this paper, there really isn't much added benefit to `flat_map` being a first-class view, so we propose to specify it as suggested above — in terms of `transform`, `join`, and possibly `cache_latest`.

§ 3.4.2 transform_maybe

For `transform_maybe`, [P1255R6] seeks to address this problem but requires using three chained adapters:

```
inline constexpr auto transform_maybe1 = [](auto&& f){
    return views::transform(FWD(f))
        | views::transform(views::maybe)
        | views::join;
};
```

This is an expensive implementation. Not only do we need three adapters, but `join` is a very complex adapter and we have an extremely specialized case here that is much simpler. Moreover, the result of `transform_maybe1` is always an input range only (we are joining a range of prvalue ranges).

We could get the same behavior out of three simpler adapters in a different way in C++20:

```
inline constexpr auto transform_maybe2 = [](auto&& f){
    return views::transform(FWD(f))
        | views::filter([](auto&& e) -> decltype(static_cast<bool>(e)) {
            return static_cast<bool>(e);
        })
        | views::transform([](auto&& e) -> decltype(decay-copy(*FWD(e))) {
            return decay-copy(*FWD(e));
        });
};
```

This implementation can now potentially be a bidirectional range (instead of input-only) thanks to avoiding `join`, but it still uses three adapters.

Both of these implementations have the issue of doing unnecessary extra copies. Let's say we have a range of `T` and a bunch of example functions, what would we want the resulting `filter_map`'s reference type to be?

function	desired reference type	maybe result	filter result
<code>T -> A*</code>	<code>A&</code>	<code>A&</code>	<code>A</code>
<code>T -> std::optional const&</code>	<code>B const&</code>	<code>B&</code>	<code>B</code>
<code>T -> std::optional<C></code>	<code>C</code>	<code>C&</code>	<code>C</code>

function	desired reference type	maybe result	filter result
T -> boost::optional<D&>	D&	D&	D

The maybe implementation always yields lvalue references, the transform-filter-transform implementation always yields prvalues. The latter is predictable - we always copy out - whereas the former doesn't need to copy the D, although it does copy the B despite perhaps looking as if it does not (it does copy the C, and needs to, although it likewise might appear as if it does not).

We can provide the desired result with a more complex version of the final transform by only decaying xvalues:

```
inline constexpr auto transform_maybe3 = [](auto&& f){
    return views::transform(FWD(f))
        | views::filter([](auto&& e) -> decltype(static_cast<bool>(e)) {
            return static_cast<bool>(e);
        })
        | views::transform([](auto&& e) requires requires { *e; } -> decltype(auto) {
            if constexpr (std::is_rvalue_reference_v<decltype(*FWD(e))>) {
                return decay-copy(*FWD(e));
            } else {
                return *FWD(e);
            }
        });
};
```

This is certainly quite a bit more complicated than the `views::tail` implementation suggested earlier!

But we don't want to specify it like that either. Even `transform3` has a huge problem: how many times do we invoke the callable? Ideally, if we're just traversing the resulting range one time, we'd invoke the function one time per element in the input range. But that's not actually the case here - we would have to end up invoking the function twice in the case that it yields an engaged `optional`. This follows directly from the underlying iterator model: the `transform_view`'s iterator `::operator*` invokes the function, while `filter_view`'s iterator `::operator++` does a `find_if`. For elements that are filtered in, there would be one dereference during the `operator++` to stop searching and then again to actually propagate the value in the `filter_view`'s iterator `::operator*`. This could be resolved by adding yet another adapter that we already mentioned:

```
inline constexpr auto transform_maybe4 = [](auto&& f){
    return views::transform(FWD(f))
        | views::cache_latest
        | views::filter([](auto&& e) -> decltype(static_cast<bool>(e)) {
            return static_cast<bool>(e);
        })
        | views::transform([](auto&& e) requires requires { *e; } -> decltype(auto) {
            if constexpr (std::is_rvalue_reference_v<decltype(*FWD(e))>) {
                return decay-copy(*FWD(e));
            } else {
                return *FWD(e);
            }
        });
};
```

We think that `transform_view` merits a first-class view to accomplish this functionality not only because the above is very involved but also because it still has unnecessary overhead - it'd be nice to avoid instantiating *four* views when one is sufficient, along with all the wrapping that entails.

§ 3.5 The sliding family

Conor talks about this family of ranges in a CppCon 2019 lighting talk [[hoekstra.cppcon](#)].

- `chunk(N)` breaks a range into non-overlapping ranges of length `N`. `views::iota(0,10) | views::chunk(4)` yields `[[0,1,2,3], [4,5,6,7], [8,9]]`. Note that the last range has length less than 4.
- `slide(N)` is very similar to `chunk` except its subranges are overlapping and all have length exactly `N`. `views::iota(0,10) | views::slide(4)` yields `[[0,1,2,3], [1,2,3,4], [2,3,4,5], [3,4,5,6], [4,5,6,7], [5,6,7,8], [6,7,8,9]]`. Note that `slide(2)` is similar to `adjacent`, except that the latter yields a range of tuples (i.e. having compile-time size) whereas here we have a range of ranges (still having runtime size). `range-v3` calls this sliding, which has a different tense from the other two, so we change it to `slide` here.
- `stride(N)` takes every `N`th element. `views::iota(0, 10) | views::stride(4)` yields `[0,4,8]`. Note that unlike the other two, this one is not a range of ranges.

These are three specific examples of a general algorithm that takes three parameters: the size of the subranges to return, the size of the step to take after each subrange, and whether to include partial ranges. Kotlin calls this algorithm `windowed`, Scala calls it `sliding`, D calls it `slide`, Haskell calls it `divvy`, and Clojure calls it `partition`.

Algorithm	Step	Size	Partial
<i>generic</i>	<code>n</code>	<code>k</code>	<code>b</code>
<code>chunk</code>	<code>k</code>	<code>k</code>	<code>true</code>
<code>slide</code>	<code>1</code>	<code>k</code>	<code>false</code>
<code>stride</code>	<code>k</code>	<code>1</code>	N/A

The above table isn't *quite* right - since `stride` does not give you a range of ranges, so it would unfortunately not be implementable in terms of *generic*. And as with `tail` vs `drop`, we have the question here of should `chunk` and `slide` each be first-class views or should they both be implemented in terms of *generic*? Implementing in terms of *generic* saves specification effort and gives us a more generic algorithm, but means we would have to store two extra data members than would be necessary in first-class implementation.

Moreover, as a language without named arguments, we have a different problem when it comes to *generic* here. It takes two `ints` and a `bool`. There is no natural order for these parameters, so who knows what `generic(1, 4, false)` means — especially since `generic(false, 1, 4)` would also compile. This suggests simply not having it as a user-facing algorithm. Or we could use an aggregate to mock up named arguments via designated-initializers, as in `generic({.step=1, .size=4, .partial=false})`. This is a very useful pattern, but one which has no precedent in the standard library as of this writing.

Ultimately, we don't think an algorithm like *generic* would necessarily be useful for C++, and it wouldn't really help much in specifying either `chunk` or `slide` (and definitely not `stride`). But these are important algorithms that come up frequently enough to be in consideration for Tier 1.

§ 3.6 The take/drop family

In C++20 already we have several views that pass through some subset of the initial range — without modifying any of the elements. Those are: `take`, `take_while`, `drop`, and `drop_while`. There are actually many more algorithms in this family that are all quite similar. Nearly all of these range adapters can be implemented in terms of adapters that already exist, although we can typically do better if we make them all first-class. The question is really what is it that we want to do here?

We already discussed the example of `tail` earlier — should this be a first-class view or is `drop(1)` sufficient? The same question applies to most of the views in this list, and we generally have the same answer for all of them: if we're okay with derivative implementations, then we might as well make all of them Tier 1 since they would all have single-sentence specifications; but if we want better implementations, then they're certainly not important enough to gain get top priority, so we would move them down to Tier 2 or Tier 3.

The potential adoption of `|>` as a language feature [P2011R1] also affects the calculus here, as most of these could also be implemented as simple functions and relying on `|>` instead of `|` to do the piping. For example, `tail` could be implemented ([P1739R4] notwithstanding):

```
template <viewable_range R>
auto tail(R&& range) {
    auto b = ranges::begin(range);
    auto e = ranges::end(range);
    if (b != e) {
        ++b;
    }
    return subrange(b, e);
}
```

We'll go through the other potential range adapters in this family and discuss how they could be implemented in terms of existing adapters:

- `take_last(N)` and `drop_last(N)`. `views::take_last(N)` is equivalent to `views::reverse | views::take(N) | views::reverse`. But this is somewhat expensive, especially for non-common views. For random-access, sized ranges, we're probably want `r | views::take_last(N)` to evaluate as `r | views::drop(r.size() - N)`, and that desire is really the crux of this whole question — is the equivalent version good enough or should we want to do it right?
- `take_last_while(P)` and `drop_last_while(P)`. These likewise could be implemented in terms of double-reverse, but `take_last_while` could not be implemented in terms of `drop_while`.
- `take_exactly(N)` and `drop_exactly(N)`. These are similar to `take` and `drop` except not checked. `iota(0, 3) | take(5)` is a valid empty range, while `iota(0, 3) | take_exactly(5)` is undefined behavior.
- `slice(M, N)` is equivalent to `views::drop(M) | views::take(N - M)`, and you couldn't do much better as a first class view. range-v3 also supports a flavor that works as `views::slice(M, end - N)` for a special variable `end`, which likewise be equivalent to `r | views::drop(M) | views::drop_last(N)`.
- `head` is equivalent to `views::drop_last(1)`.
- `tail` is equivalent to `views::drop(1)`.
- `trim(P)` is equivalent to `views::drop_while(P) | views::drop_last_while(P)`.
- `delimit(V)` has two flavors. When called with a range, `r | views::delimit(V)` is equivalent to `r | views::take_while([V](auto&& e) { return !(e == V); })`. But it also allows wrapping an iterator, such that `views::delimit(it, V)` is equivalent to `subrange(it, unreachable_sentinel) | views::delimit(V)`.

We're tentatively labelling the more complicated ones here Tier 2 and the more trivial (i.e. `head`, `tail`, and `slice`) and possibly less needed (i.e. `meow_exactly`) ones Tier 3.

§ 3.7 Generative factories

There are several views on the list that are simply factories — they cannot be piped into. So we'll consider them as their own family:

- `cartesian_product(E...)` takes a bunch of ranges and yields a range of tuples that are the Cartesian product of those ranges.
- `concat(E...)` concatenates a bunch of ranges together, it must be a standalone range. It is also sometimes called `chain`.
- `generate(F)` takes a nullary function `F` and produces an infinite range of invoking that function.
- `generate_n(F, N)` is equivalent to `generate(F) | views::take(N)`.

- `linear_distribute(B, E, N)` produces a range of `N` values linearly distributed from `B` to `E`.
- `repeat(V)` is an infinite range of a single value, equivalent to `generate([V]{ return V; })`.
- `repeat_n(V, N)` is `N` copies of `V`, equivalent to `generate_n([V]{ return V; }, N)`.

These vary wildly in complexity (`repeat` is certainly far simpler than `cartesian_product`). But `cartesian_product` comes up sufficiently often and are sufficiently complicated to merit Tier 1 priority.

The rest, we consider lower priority. And `generate` and `generate_n` in particular need special care to deal with 16.4.6.10 [\[res.on.data.races\]](#). The current implementation of `generate_n` in `range-v3` has a data race.

§ 3.8 Other view adapters

Other range adapters that we haven't talked about yet, but aren't sure how to group exactly, are:

- `adjacent_filter(P)` is an extension of `filter` that rather than taking a unary predicate and going one element at a time takes a binary predicate and goes pairwise through the elements. Similar to `adjacent | filter(applied(P)) | views::keys` except that it always keeps the first element.
- `adjacent_remove_if(P)` is similar to `adjacent_filter(not_fn(P))`, except that it always keeps the last element.
- `cycle(R)` produces an infinite ranges that, well, cycles through `R` repeatedly.
- `scan(R, F, V)` is the lazy view version of `std::inclusive_scan`, except not having a defaulted binary operation.
- `intersperse(V)` produces a new range alternating selecting elements from the source range and the value `V`
- `join_with(V)`. C++20 has a version of `join` that does not take a delimiter, but we really do need a version that provides one as well. The issue with taking a delimiter is that there is an ambiguity with what `r | views::join(v)` means, if `v` happens to itself be a joinable range. `range-v3` assumes that if `v` is a joinable range that `views::join(v)` joins it without a delimiter. We think this ship has sailed in C++20, and it would be better to introduce `join_with` that requires a delimiter.
- `partial_sum(R)` is equivalent to `scan(R, std::plus<>())`.
- `split_when(P)` is a more complicated version of `split` that rather than taking a value or a range, instead takes a predicate that could also return an iterator.
- `sample(N)` yields a random sample of the given length.

There are other combinatoric generators that also could be explored. For example, Python has `itertools.product`, `itertools.combinations`, and `itertools.combination_with_replacement` which all operate on a single range of τ and produce a range of range of τ .

Of these, `views::join_with` fills in an incomplete aspect of the already-existing `views::join`, so we feel that it is a Tier 1 view. The rest we consider to have lower priority.

§ 3.9 Derivatives of transform

Several of the above views that are labeled “not proposed” are variations on a common theme: `addressof`, `indirect`, and `move` are all basically wrappers around `transform` that take `std::addressof`, `std::dereference` (a function object we do not have at the moment), and `std::move`, respectively. Basically, but not exactly, since one of those functions doesn't exist yet and the other three we can't pass as an argument anyway.

But some sort of broader ability to pass functions into functions would mostly alleviate the need for these.

`views::addressof` is shorter than `views::transform(LIFT(std::addressof))` (assuming a `LIFT` macro that wraps a

name and emits a lambda), but we're not sure that we necessarily need to add special cases of transform for every useful function.

§ 3.9.1 `views::const_`

There's one that stands out as being slightly different though: `views::const_` isn't quite the same as a transform over `std::as_const`, because of the existence of proxy references. `std::as_const` only accepts lvalues, so something like:

```
std::vector<int> v;  
views::zip(v) | views::transform(LIFT(std::as_const));
```

would not compile. And were it to compile, what should it actually mean? `zip`'s reference type here, as previously discussed, should be `std::tuple<int&>`. What would a `const` view over this range mean? Should we transparently propagate prvalues, thereby being a shallow `const`, and produce a new range whose reference is `std::tuple<int&>`? Or should we somehow come up with a way to do deep `const`-ness here and yield a range whose reference is `std::tuple<int const&>`? The latter noting that we are not doing this sort of introspection for `views::zip_transform`.

In range-v3, the reference type is `common_reference_t<range_value_t<R> const&&, range_reference_t<R>>`. In this particular example, that would be `std::tuple<int> const`. This does not seem like a good choice, since it would involve actually copying elements. Maybe not terrible here because they're `ints`, but for more expensive types it would be very surprising if `views::const_` actually incurred overhead.

Having a `const` view over a range is something that seems inherently useful and is more complex than simply a transform over `std::as_const`, but is something that needs more research. As such, we think it's Tier 3 material.

§ 4 Algorithms

The largest chunk of C++20 Ranges were the algorithms, and the work here has been very thorough. All the `<algorithm>`s have been range-ified, which has been fantastic.

But there are a few algorithms that aren't in `<algorithm>` that do not have range-based versions: those that are in `<numeric>`. These are often the last algorithms considered for anything, they were the last algorithms that were made `constexpr` ([P1645R1]) and now are the last to become range-based. They are:

Algorithm	Priority
<code>iota</code>	Tier 1
<code>accumulate</code>	Tier 1, renamed to <code>fold</code> .
<code>reduce</code>	Tier 2, along with <code>sum</code> and <code>product</code> .
<code>transform_reduce</code>	Not proposed.
<code>inner_product</code>	Not proposed.
<code>adjacent_difference</code>	Tier 3, renamed to <code>adjacent_transform</code>
<code>partial_sum</code>	Tier 3, but without a binary operation parameter. Also adding <code>partial_fold</code> .
<code>inclusive_scan</code>	Tier 3
<code>exclusive_scan</code>	Tier 3
<code>transform_inclusive_scan</code>	Not proposed.
<code>transform_exclusive_scan</code>	Not proposed.

What to do about these algorithms? Well, one of the big motivations for Ranges was the ability to actually compose algorithms. This severely reduces the need for the combinatorial explosion of algorithms - all the `transform_meow`

algorithms are transform followed by meow, so we probably don't need separate range-based algorithms for those.

Four of these (`accumulate`, `reduce`, `transform_reduce`, and `inner_product`) return a value, while the other seven output a range (one through a pair of writable iterators and the other six through an output iterator). We'll consider these separately.

§ 4.1 Algorithms that Output a Value (Catamorphisms)

§ 4.1.1 `std::accumulate` → `ranges::fold`

We think having a range-based left-fold algorithm in the standard library is very important, since this is such a fundamental algorithm. Indeed, several of the other standard library algorithms *are* simple folds — for instance `count`, `count_if`, `max_element`, `min_element`, `minmax_element`, and `inner_product`. We don't have a generic range-based `max` or `min` (just ones that takes an `initializer_list`), but those would also be a left-folds. As such, we think adding such a left-fold to the standard library is a top tier priority for C++23. Except that we think this algorithm should be named `ranges::fold` - the problem with the name `accumulate` is that it is strongly suggestive of addition, which makes uses of it over different operations just very strange. `fold` is what the algorithm is, and has no such emphasis. It's the more generic name, for the most generic algorithm.

[P1813R0] goes through the work of introducing a set of constraints for these algorithms, and its suggestion for this algorithm is:

```
template <input_range R, movable T, class Proj = identity,
         indirect_magma<const T*, projected_iterator_t<R>, Proj>, T*> BOp = ranges::plus>
constexpr accumulate_result<safe_iterator_t<R>, T>
    accumulate(R&& r, T init, BOp bop = {}, Proj proj = {});
```

We think this is a bad direction, for three reasons.

First, we should not default the binary operation at all. Having a default `fold` operation doesn't make much sense - it's reasonable for `ranges::sort` to default to sorting by `<`, since the entire standard library is built on `<` as the primary comparison operator, but that doesn't really hold for `+`. Instead, we should add separate named algorithms `ranges::sum` and `ranges::product` that just invoke `ranges::fold` with `std::plus()` and `std::multiplies()` – or more likely that these invoke `ranges::reduce` instead as the more efficient algorithm with more constraints.

Second, the above definition definitely follows Alexander Stepanov's law of useful return [stepanov] (emphasis ours):

When writing code, it's often the case that you end up computing a value that the calling function doesn't currently need. Later, however, this value may be important when the code is called in a different situation. In this situation, you should obey the law of useful return: *A procedure should return all the potentially useful information it computed.*

But it makes the usage of the algorithm quite cumbersome. The point of a fold is to return the single value. We would just want to write:

```
int total = ranges::sum(numbers);
```

Rather than:

```
auto [_, total] = ranges::sum(numbers);
```

or:

```
int total = ranges::sum(numbers, 0).value;
```


`ranges::fold` should just return `T`. This would be consistent with what the other range-based folds already return in C++20 (e.g. `ranges::count` returns a `range_difference_t<R>`, `ranges::any_of` - which can't quite be a fold due to wanting to short-circuit - just returns `bool`).

Third, these constraints are far too restrictive. Copying the proposed definition of `magma` and `indirect_magma` here for readability:

```
// section 3.2.2 from P1813R0
template <class BOp, class T, class U>
concept magma =
    common_with<T, U> &&
    regular_invocable<BOp, T, T> &&
    regular_invocable<BOp, U, U> &&
    regular_invocable<BOp, T, U> &&
    regular_invocable<BOp, U, T> &&
    common_with<invoke_result_t<BOp&, T, U>, T> &&
    common_with<invoke_result_t<BOp&, T, U>, U> &&
    same_as<invoke_result_t<BOp&, T, U>, invoke_result_t<BOp&, U, T>>;
```

Let `bop` be an object of type `BOp`, `t` be an object of type `T`, and `u` be an object of type `U`. The value `invoke(bop, t, u)` must return a result that is representable by `common_type_t<T, U>`.

The decision to require common types for a over `magma<T, U>` is similar to the reason that `equality_comparable_with` requires `common_reference_with`: this ensures that when an algorithm requires a `magma`, we are able to *equationally reason* about those requirements. It's possible to overload `operator+(int, vector<int> const&)`, but that doesn't follow the canonical usage of `+`. Does `1 + vector{1, 2, 3}` mean "concatenate `vector{1, 2, 3}` to the end of a temporary `vector{1}`"? Is it a shorthand for `accumulate(vector{1, 2, 3}, 1)`? The intention is unclear, and so `std::plus<>` should not model `magma<int, vector<int>>`.

```
// section 3.2.11 from P1813R0
template <class BOp, class I1, class I2, class O>
concept indirect_magma =
    readable<I1> &&
    readable<I2> &&
    writable<O, indirect_result_t<BOp&, I1, I2>> &&
    magma<BOp&, iter_value_t<I1>&, iter_value_t<I2>&> &&
    magma<BOp&, iter_value_t<I1>&, iter_reference_t<I2>&> &&
    magma<BOp&, iter_reference_t<I1>, iter_value_t<I2>&> &&
    magma<BOp&, iter_reference_t<I1>, iter_reference_t<I2>> &&
    magma<BOp&, iter_common_reference_t<I1>, iter_common_reference_t<I2>>>;
```

We see here again the heavy association of `plus` with `accumulate`, hence again the desire to rename the algorithm to `fold`. But the important thing to consider here is the requirement that the binary function *need* be invocable on each type and that there *need* be a common type for the result. We've already been through this process with the ranges comparison algorithms in [P1716R3] and removed those restrictions.

Consider a simple fold counting the occurrences of a string (i.e. how you would implement `ranges::count` with `ranges::fold`):

```
std::vector<std::string> words = /* ... */;
int n = ranges::fold(words, 0, [](int accum, std::string const& w){
    return accum + (w == "range");
});
```

Such an algorithm would not meet the requirements laid out in P1813. There's no common type between `int` and `string`, that lambda is only invocable with one of the possible four orders of arguments. But it's a perfectly reasonable

fold. Instead, the only allowed implementation would be:

```
int n = ranges::fold(words, 0, ranges::plus{}, [](std::string const& w) { return w == "ranges";
});
```

But we're hard-pressed to explain why this would be considered better. In the general case, there may not even be an allowed implementation. Consider wanting to score a word in Scrabble. In Scrabble, each letter has a value but each tile can either multiply the score of a single letter or multiple the score of the whole word. One way to compute the score then is to use two folds, one to figure out the word multiplier and another to figure out the letter sum:

```
struct Square { int letter_multiplier, word_multiplier; };
vector<Square> squares = /* ... */;
vector<int> letters = /* ... */;

int score = fold(squares, 1, multiplies(), &Square::word_multiplier)
    * fold(zip_transform(multiplies(),
        squares | views::transform(&Square::letter_multiplier),
        letters),
        0, plus());
```

Another way is to keep a running sum of the two parts separately, and do a manual multiply:

```
struct Accum {
    int result() const { return total * multiplier; };
    int multiplier, total;
};

int score = fold(zip(squares, letters), Accum(), [](Accum a, auto const& sq_let){
    auto [sq, letter] = sq_let;
    return Accum{
        .multiplier = a.multiplier * sq.word_multiplier,
        .total = a.total + sq.letter_multiplier * letter
    };
}).result();
```

We're not trying to argue that the second solution is necessarily better than the first - merely that it is a perfectly adequate solution, that happens to not be able to meet the constraints as laid out in P1813.

Instead, we suggest a much lighter set of restrictions on fold: simply that this is a binary operation:

```
template <class F, class T, class U>
concept foldable =
    regular_invocable<F&, T, U> &&
    convertible_to<invoke_result_t<F&, T, U>, T>;

template <class F, class T, class I>
concept indirectly-binary-foldable =
    indirectly_readable<I> &&
    copy_constructible<F> &&
    foldable<F, T, iter_value_t<I>> &&
    foldable<F, T, iter_reference_t<I>> &&
    foldable<F, T, iter_common_reference_t<I>>;

template <input_range R, movable T, class Proj = identity,
    indirectly-binary-foldable<T, projected<iterator_t<R>, Proj>> BinaryOperation>
constexpr T fold(R&& r, T init, BinaryOperation op, Proj proj = {}) {
    range_iterator_t<R> b = begin(r);
    range_sentinel_t<R> e = end(r);
    for (; b != e; ++b) {
        init = op(std::move(init), proj(*b));
    }
}
```

```
    return init;
}
```

§ 4.1.2 `ranges::reduce`

We have this interesting situation in the standard library today where `std::accumulate` has a name strongly suggestive of addition, yet because it's specified to invoke its binary operation serially, it has no additional requirements on that operation. But we also have `std::reduce`, which is a much more generic name with no suggested underlying operation, yet has very strong semantic constraints on its operation: it must be both associative and commutative. This comes from [N3408], emphasis ours:

Thrust has no `accumulate` algorithm. Instead, it introduces the analogous `thrust::reduce`, **which requires stricter semantics from its user-specified sum operator to allow a parallel implementation**. Specifically, `thrust::reduce` requires mathematical associativity and commutativity of its user-specified sum operator. This allows the algorithm implementor discretion to parallelize the sum. We chose the name `reduce` for this algorithm because we believe that most existing parallel programmers are familiar with the idea of a parallel reduction. Other names for this algorithm exist, e.g., `fold`. However, we did not select a name derived from `fold` because other languages tend to impose a non-associative directionality to the operation. [cf. Haskell's `foldl` & `foldr`, Scala's `foldLeft` & `foldRight`]

While `ranges::fold` should have minimal constraints, that is not the case for a future `ranges::reduce`. As with `std::reduce`, we would need to enforce that the binary operation is both associative and commutative. This calls for the kinds of constraints that [P1813R0] is proposing. As it is a more complex set of constraints, we suggest that this is a **Tier 2 algorithm**, with no default operation. Given the previous suggestion of `ranges::fold` not having a default operation either, we also suggest the addition of a **`ranges::sum` and a `ranges::product`** that simply invoke `ranges::reduce` with `std::plus()` and `std::multiplies()`, respectively, with an initial value defaulted to `range_value_t<R>()` and `range_value_t<R>{1}`, respectively.

The naming here is somewhat problematic. `reduce` is, in general, a much better name than `accumulate` as it does not have any particular operation connotation. But it has additional requirements on the operation. With the suggested change in name, we would end up having both `fold` and `reduce` — names that seem synonymous and interchangeable, though they are not. We feel that this is probably okay though, since people already frequently think `reduce` is “just” the parallel version of `accumulate` and perhaps having `fold` and `reduce` both would make users more likely to consult the documentation?

§ 4.1.3 `ranges::transform_reduce` and `ranges::inner_product`

These two algorithms are different from the previous two in that they are less fundamental. `transform_reduce` is a binary transform followed by `reduce` while `inner_product` is a binary transform followed by `accumulate`. First, `inner_product`, much like `accumulate`, is a bad name for the algorithm as it strongly prejudices `product` as the binary transform operation and as such uses of the algorithm with any other function simply look bizarre. From the [cppreference example for inner_product](#):

```
#include <numeric>
#include <iostream>
#include <vector>
#include <functional>
int main()
{
    std::vector<int> a{0, 1, 2, 3, 4};
    std::vector<int> b{5, 4, 2, 3, 1};

    int r1 = std::inner_product(a.begin(), a.end(), b.begin(), 0);
    std::cout << "Inner product of a and b: " << r1 << '\n';
}
```

```

int r2 = std::inner_product(a.begin(), a.end(), b.begin(), 0,
                           std::plus<>(), std::equal_to<>());
std::cout << "Number of pairwise matches between a and b: " << r2 << '\n';
}

```

Second, and more importantly, with Ranges allowing us to compose algorithms properly, do we even need these at all? Consider again the above example and how it might be written with and without specialized algorithms:

Specialized	Composed
<pre> ranges::inner_product(a, b, 0, std::plus(), std::equal_to()); </pre>	<pre> ranges::fold(views::zip_transform(std::equal_to(), a, b), 0, std::plus()); ranges::sum(views::zip_transform(std::equal_to(), a, b)); </pre>

Even though the `ranges::fold` construction is more complicated, it's also easier to see the groupings and understand what's going on. The composed construction also allows for arbitrarily many ranges, not simply two.

There is also the question of projections. With `transform_reduce` and `inner_product`, there are *three* ranges that could be projected: each range into the binary grouping operation, and the result of that grouping. This makes it exceedingly awkward if you only want to provide exactly one of those projections:

Specialized	Composed
<pre> ranges::inner_product(a, b, 0, std::plus(), std::multiplies(), p1); </pre>	<pre> ranges::fold(views::zip_transform(std::multiplies(), a views::transform(p1), b), 0, std::plus()); </pre>
<pre> ranges::inner_product(a, b, 0, std::plus(), std::multiplies(), {}, p2); </pre>	<pre> ranges::fold(views::zip_transform(std::multiplies(), a, b views::transform(p2)), 0, std::plus()); </pre>
<pre> ranges::inner_product(a, b, 0, std::plus(), std::multiplies(), {}, {}, p3); </pre>	<pre> ranges::fold(views::zip_transform(std::multiplies(), a, b) views::transform(p3), 0, std::plus()); </pre>

We think that once we add `ranges::fold` as Tier 1 and `ranges::reduce` as Tier 2, we do not actually have a need for either a `ranges::transform_reduce` or a `ranges::inner_product` (which would also save us from having to come up with a name for the latter).

§ 4.2 Algorithms that Output a Range (Anamorphisms)

`iota` is the easiest one to consider here. We already have `views::iota` in C++20, which importantly means that we already have all the correct constraints in place. In that sense, it almost takes less time to adopt `ranges::iota` than it would take to discuss whether or not it's worth spending time adopting it.

But that does not hold for the other algorithms.

§ 4.2.1 `std::adjacent_difference` → `ranges::adjacent_transform`

`std::adjacent_difference` joins `std::accumulate` and `std::inner_product` in the list of algorithms prejudicially named after a specific operation. We do not yet have `views::adjacent_transform` ([Tier 1](#) above), and this would be the algorithm version of those views:

Specialized	Composed
<code>ranges::adjacent_difference(r, o);</code>	<code>ranges::copy(views::adjacent_transform(r, std::minus()), o);</code>
<code>ranges::adjacent_difference(r, o, f);</code>	<code>ranges::copy(views::adjacent_transform(r, f), o);</code>

Even though we're increasing the length of the expression as we go, and arguably increasing the complexity of the construction as well, we're also lowering the surface area of the API by taking advantage of composition. These become even better with the adoption of the pipeline operator in [\[P2011R1\]](#):

Specialized	Composed
<code>ranges::adjacent_difference(r, o);</code>	<code>views::adjacent_transform(r, std::minus()) > ranges::copy(o);</code>
<code>ranges::adjacent_difference(r, o, f);</code>	<code>views::adjacent_transform(r, f) > ranges::copy(o);</code>

This begs the question: do we actually need to have a `ranges::adjacent_transform` at all? This question needs to be answered, and its existence lowers the priority of the range-ification of such algorithms relative to the adoption of their corresponding range adapters.

§ 4.2.2 `std::partial_sum` → `ranges::partial_fold` and `std::{in,ex}clusive_scan`

We saw in the catamorphism section that we have a pair of algorithms, `std::accumulate` and `std::reduce`, that solve basically the same problem except that one prejudices a particular operation (`std::accumulate` suggests `+`) while the other has the more generic name yet is actually more restrictive (`std::reduce` requires both the operation to be both associative and commutative, `std::accumulate` does not require either).

We have the exact same issue here, `std::partial_sum` is strongly suggestive of `+`, while `std::inclusive_scan` is the more generically-named algorithm that nevertheless imposes the stronger restriction (in this case, just associativity).

Our suggestion for what to do with `std::partial_sum` and `std::{in,ex}clusive_scan` thus mirrors our suggestion for what we did with `std::accumulate` and `std::reduce`:

- rename `std::partial_sum` to `ranges::partial_fold` (since it's a fold that also yields partial results), which will have neither a defaulted binary operation nor associativity requirements.
- introduce `ranges::{in,ex}clusive_scan`
- introduce `ranges::partial_sum` that is hard-coded to use `std::plus()` as the binary operation, which internally forwards to `ranges::inclusive_scan` (not `ranges::partial_fold`, since we know addition is associative).

As we discussed with the question of the need for `adjacent_difference`, there would also be the question of whether we need these algorithms at all. As such, we ascribe them fairly low priority.

§ 4.2.3 `transform_{in,ex}clusive_scan`

Similar to the question of `transform_reduce` and `inner_product`, we don't think we need a:

```
ranges::transform_inclusive_scan(r, o, f, g);
```

once we can write

```
ranges::inclusive_scan(r | views::transform(g), o, f);
```

or even

```
ranges::copy(r | views::transform(g) | views::inclusive_scan(f), o);
```

The latter two having the nice property that you don't have to remember the order of operations of the operations. We don't think we need these at all.

§ 4.3 Parallel Algorithms

One of the C++17 additions was the introduction of the parallel algorithms by way of the Parallelism TS [P0024R2]. But with Ranges, we don't have parallel overloads of any of the algorithms yet. How should we prioritize adding parallel overloads for the range algorithms?

The most important issue to consider is: with all the ongoing work on executors, we very much want to ensure that the parallel overloads we define will end up working. The status quo in the standard library is that the `ExecutionPolicy` parameter is constrained on `is_execution_policy_v<remove_cvref_t<ExecutionPolicy>>`. Is that good enough for executors? It might not be, and it would be extremely disappointing to adopt executors in a way that is incompatible with a bunch of parallel algorithms we just added. This needs careful consideration by somebody familiar with executors.

The second issue is that there are additional requirements imposed on the parallel overloads as compared to the sequential ones (see 25.3 [algorithms.parallel] as well as [P0836R1]). Those requirements would need to be somehow captured in concepts. Even if the type trait is considered sufficient for executors going forward, the concepts work is still necessary.

And for those algorithms which we do not yet have range-based overloads, we still have exactly the same issue for why we don't have range-based overloads yet: what are the concepts that we need to constrain the various parameters?

Due to the executor dependency and the need to be careful about specifying the concepts for parallel requirements, we consider this Tier 2.

§ 5 Actions

There are really three kinds of operations that exist in range-v3:

- Algorithms are operations that eagerly either mutate an input range (e.g. `sort`), produce a value (e.g. `max_element`), both (partition), or produce a new range by way of an output iterator or output range (e.g. `copy`).
- Views are operations that lazily produce a new range.
- Actions are operations that eagerly produce a new range.

While we have range-based algorithms and views in C++20, we do not yet have any actions. range-v3 comes with many actions, some of which are the action flavor of views and some of which are the action flavor of algorithms. The full list is: `drop`, `drop_while`, `erase`, `insert`, `join`, `push_back`, `push_front`, `remove_if`, `remove`, `reverse`, `shuffle`, `slice`, `sort`, `split`, `stable_sort`, `stride`, `take`, `take_while`, `transform`, `unique`, and `unstable_remove_if`.

The advantage of these actions is that unlike the algorithms, they compose, and unlike the views, you can pass rvalue ranges to them. From the examples in the range-v3 documentation [[range-v3.docs](#)]:

When you want to mutate a container in-place, or forward it through a chain of mutating operations, you can use actions. The following examples should make it clear.

Read data into a vector, sort it, and make it unique.

```
extern std::vector<int> read_data();
using namespace ranges;
std::vector<int> vi = read_data() | actions::sort | actions::unique;
```

Do the same to a vector that already contains some data:

```
vi = std::move(vi) | actions::sort | actions::unique;
```

Mutate the container in-place:

```
vi |= actions::sort | actions::unique;
```

Same as above, but with function-call syntax instead of pipe syntax:

```
actions::unique(actions::sort(vi));
```

Indeed, ranges actions do seem very useful — and provide the ability to pipeline operations that we cannot do today. There is, nor will there ever be, a `views::sort`, so the only option today is:

```
std::vector<int> vi = read_data();
ranges::sort(vi);
ranges::unique(vi);
```

But while it might be nicer to provide a pipeline approach, we feel this whole space simply needs more research. It is an immediate source of frustration for users when they discover that while you can pipe a view into an action, you cannot do the reverse.

Given that the actions don't provide any functionality that we don't already have, simply adding the ability to compose some operations better, we give them pretty low priority relative to the wealth of new functionality many of the other operations here provide.

§ 6 Plan Summary

To summarize the above descriptions, we want to triage a lot of outstanding ranges algorithms, views, actions, and other utilities into three tiers based on our opinions of their importance. While ideally we could just add everything into C++23, we realize that this is not realistic with the amount of available LWG bandwidth, so our tier 1 here is trying to be as small as possible while still hitting as many major pain points as possible.

§ 6.1 Tier 1

- `ranges::to`
- the ability to format views with `std::format`
- the addition of the following first class range adapters:
 - `views::cache_latest`
 - `views::cartesian_product`

- `views::chunk`
- `views::group_by`
- `views::iter-zip-transform<V>` (exposition-only)
- `views::iter-adjacent-transform<V>` (exposition-only)
- `views::index-view<S, D>` (exposition-only)
- `views::join_with`
- `views::slide`
- `views::stride`
- `views::transform_maybe`
- the addition of the following range adapters specified in terms of other range adapters:
 - `views::enumerate`
 - `views::flat_map` (renamed to... something)
 - `views::zip`
 - `views::zip_transform`
 - `views::adjacent`
 - `views::adjacent_transform`
- the addition of the following range algorithms:
 - `ranges::iota`
 - `ranges::fold`
- the following other changes to standard library (necessary for the zip family):
 - `pair<T, U>` should be const-assignable whenever `T` and `U` are both const-assignable
 - `pair<T&, U&>` should be constructible from `pair<T, U>&`
 - `tuple<T...>` should be const-assignable whenever `T...` are const-assignable
 - `tuple<T&...>` should be constructible from `tuple<T...>&`.
 - `vector<bool>::reference` should be const-assignable
 - `bitset<N>::reference` should be const-assignable

§ 6.2 Tier 2

- the addition of the following range adapters:
 - `views::concat`
 - `views::cycle`
 - `views::delimit`
 - `views::drop_last`
 - `views::drop_last_while`
 - `views::generate`
 - `views::generate_n`

- `views::group_by_key`
- `views::intersperse`
- `views::partial_sum`
- `views::remove`
- `views::remove_if`
- `views::repeat`
- `views::repeat_n`
- `views::replace`
- `views::replace_if`
- `views::scan`
- `views::split_when`
- `views::take_last`
- `views::take_last_while`
- `views::trim`
- `views::unique`
- the addition of the following range algorithms:
 - parallel overloads of all the existing algorithms in `std::ranges` that have a parallel overload in `std`
 - `ranges::reduce`
 - `ranges::sum`
 - `ranges::product`

§ 6.3 Tier 3

- the addition of the following range adapters:
 - `views::adjacent_filter`
 - `views::adjacent_remove_if`
 - `views::const_`
 - `views::drop_exactly`
 - `views::head`
 - `views::linear_distribute`
 - `views::sample`
 - `views::slice`
 - `views::tail`
 - `views::take_exactly`
- the addition of the following range algorithms:
 - `ranges::adjacent_transform`
 - `ranges::partial_fold`

- `ranges::inclusive_scan`
- `ranges::exclusive_scan`
- `ranges::partial_sum`
- the addition of ranges actions

§ 7 References

- [hoekstra.cppcon] Conor Hoekstra. 2019. 23 Ranges: slide & stride.
<https://www.youtube.com/watch?v=-lqZJK2vjI>
- [N3408] J. Hoberock, O. Giroux, V. Grover, J. Marathe, et al. 2012. Parallelizing The Standard Algorithms Library.
<https://wg21.link/n3408>
- [niebler.iter.0] Eric Niebler. 2015. To Be or Not to Be (an Iterator).
<http://ericniebler.com/2015/01/28/to-be-or-not-to-be-an-iterator/>
- [niebler.iter.1] Eric Niebler. 2015. Iterators++, Part 1.
<http://ericniebler.com/2015/02/03/iterators-plus-plus-part-1/>
- [niebler.iter.2] Eric Niebler. 2015. Iterators++, Part 2.
<http://ericniebler.com/2015/02/13/iterators-plus-plus-part-2/>
- [niebler.iter.3] Eric Niebler. 2015. Iterators++, Part 3.
<http://ericniebler.com/2015/03/03/iterators-plus-plus-part-3/>
- [P0024R2] Jared Hoberock. 2016. The Parallelism TS Should be Standardized.
<https://wg21.link/p0024r2>
- [P0592R4] Ville Voutilainen. 2019. To boldly suggest an overall plan for C++23.
<https://wg21.link/p0592r4>
- [P0836R1] Gordon Brown, Christopher Di Bella, Michael Haidl, Toomas Remmelg, Ruyman Reyes, Michel Steuwer, Michael Wong. 2018. Introduce Parallelism to the Ranges TS.
<https://wg21.link/p0836r1>
- [P0896R4] Eric Niebler, Casey Carter, Christopher Di Bella. 2018. The One Ranges Proposal.
<https://wg21.link/p0896r4>
- [P1206R1] Corentin Jabot, Eric Niebler, Casey Carter. 2019. `ranges::to`: A function to convert any range to a container.
<https://wg21.link/p1206r1>
- [P1255R6] Steve Downey. 2020. A view of 0 or 1 elements: `views::maybe`.
<https://wg21.link/p1255r6>
- [P1522R1] Eric Niebler. 2019. Iterator Difference Type and Integer Overflow.
<https://wg21.link/p1522r1>
- [P1645R1] Ben Deane. 2019. `constexpr` for numeric algorithms.
<https://wg21.link/p1645r1>
- [P1716R3] Tomasz Kamiński. 2019. ranges compare algorithm are over-constrained.
<https://wg21.link/p1716r3>

- [P1739R4] Hannes Hauswedell. 2020. Avoid template bloat for `safe_ranges` in combination with “subrange-y” view adaptors.
<https://wg21.link/p1739r4>
- [P1813R0] Christopher Di Bella. 2019. A Concept Design for the Numeric Algorithms.
<https://wg21.link/p1813r0>
- [P1894R0] Andrew Tomazos. 2019. Proposal of `std::upto`, `std::indices` and `std::enumerate`.
<https://wg21.link/p1894r0>
- [P2011R1] Barry Revzin, Colby Pike. 2020. A pipeline-rewrite operator.
<https://wg21.link/p2011r1>
- [P2164R1] Corentin Jabot. 2020. `views::enumerate`.
<https://wg21.link/p2164r1>
- [P2210R0] Barry Revzin. 2020. Superior String Splitting.
<https://wg21.link/p2210r0>
- [range-v3] Eric Niebler. 2014. range-v3.
<https://github.com/ericniebler/range-v3/>
- [range-v3.1141] voivoid. 2019. `view::enumerate` issues with latest 1.0-beta commits?
<https://github.com/ericniebler/range-v3/issues/1141>
- [range-v3.573] Jarod42. 2017. Readable types with prvalue reference types erroneously model `IndirectlyMovable`.
<https://github.com/ericniebler/range-v3/issues/573>
- [range-v3.docs] Eric Niebler. 2014. range-v3 documentation.
<https://ericniebler.github.io/range-v3/>
- [stepanov] Alexander A. Stepanov. 2014. From Mathematics to Generic Programming.
- [stl2.381] Eric Niebler. 2017. Readable types with prvalue reference types erroneously model `Writable`.
<https://github.com/ericniebler/stl2/issues/381>
- [swift.algorithms] Nate Cook. Announcing Swift Algorithms.
<https://swift.org/blog/swift-algorithms/>